# REDIS HANDBOOK

@Aman_Views

## Redis From Zero To Hero Notes

### 1. WHAT IS REDIS?

Redis (REmote DIctionary Server) is an open-source, in-memory data structure store used as a database, cache, message broker and more.

- In-memory Key-Value store
- Extremely fast
- Supports rich data structures
- Open source & very simple to use

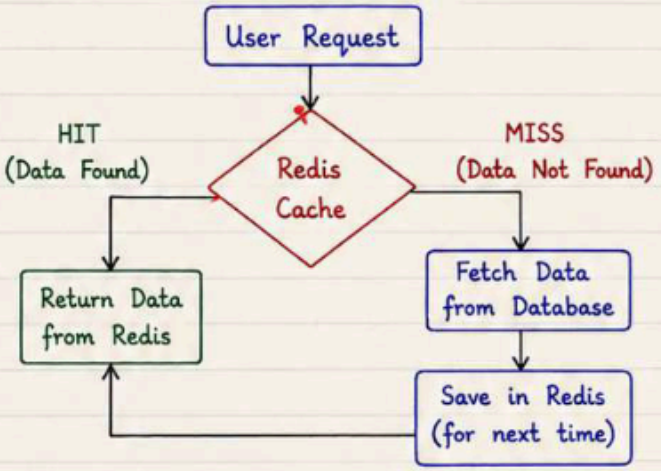
### 2. WHY REDIS?

- ✓ Blazing fast (in-memory)
- ✓ High performance
- ✓ Supports many data structures
- ✓ Easy to use
- ✓ Scalable
- ✓ Reliable (Replication, Persistence)
- ✓ Widely used in real-world systems

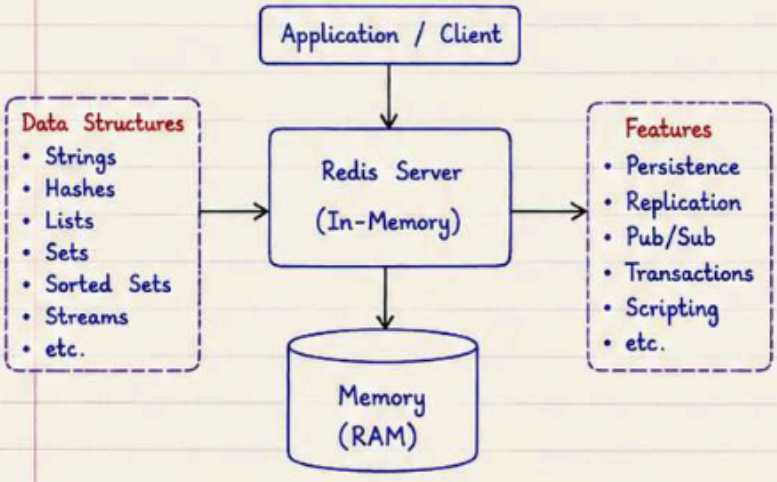
### 3. PROBLEMS WITHOUT REDIS

- ✗ Repeated database queries increase load.
- ✗ Slow response time.
- ✗ High latency for frequently accessed data.
- ✗ Difficult to handle high traffic.
- ✗ No efficient caching mechanism.
- ✗ Poor scalability.

### 4. REAL-WORLD CACHE EXAMPLE




### 5. REDIS ARCHITECTURE




### 6. COMPANIES USING REDIS

Netflix | Uber | Instagram

Amazon | Swiggy | Zomato

GitHub | StackOverflow | Shopify

Twitter(X) | Medium | Many More...

# 2. REDIS DATA STRUCTURES

Redis supports multiple data structures. Each one is optimized for specific use cases.

| # | DATA TYPE | WHAT IS IT? | WHEN TO USE? | EXAMPLE COMMAND | SAMPLE EXAMPLE |
|---|---|---|---|---|---|
| 1. | String (Key-Value) | A simple Key-Value pair. Value can be string or number. | • Cache<br>• Counters<br>• Session ID<br>• Flags | SET name Aman<br>GET name | Key : name<br>Value : Aman |
| 2. | Hash (Field-Value) | A collection of field-value pairs stored under one key. | • User Profile<br>• Product Info<br>• Settings | HSET user:1 name Aman<br>HSET user:1 age 25<br>HGETALL user:1 | user:1<br>name : Aman<br>age : 25<br>email : a@x.com<br>... |
| 3. | List (Ordered List) | A list of strings in insertion order. (Duplicates allowed) | • Queue<br>• Stack<br>• Message List<br>• Activity Feed | LPUSH mylist A<br>LPUSH mylist B<br>LRANGE mylist 0 -1 | mylist<br>B \| A<br>(Head) (Tail) |
| 4. | Set (Unordered Set) | A collection of unique strings. (No duplicates) | • Unique Users<br>• Tags<br>• Friends<br>• Remove duplicates | SADD myset A B C<br>SMEMBERS myset | myset<br>A \| B \| C |
| 5. | Sorted Set (ZSet) | A set of unique strings with a score. Sorted by score. | • Leaderboard<br>• Ranking<br>• Top N items<br>• Priority Queue | ZADD rank 100 Aman<br>ZADD rank 80 John<br>ZREVRANGE rank 0 -1 WITHSCORES | rank (High → Low)<br>Aman 100<br>John 80<br>... |
| 6. | Bitmap (Bits) | Stores bits (0 or 1). Very space efficient. | • User online status<br>• Feature Flags<br>• Analytics | SETBIT signups 100 1<br>GETBIT signups 100 | Bit at 100 = 1 |
| 7. | HyperLogLog (Approx Count) | Probabilistic data structure used for counting unique items. | • Unique Visitor Count<br>• Analytics | PFADD hll A B C<br>PFCOUNT hll | Approx Count = 12500 |
| 8. | Stream (Log Stream) | A log-like data structure for streaming data. | • Event Streaming<br>• Activity Logs<br>• Queue / Pipeline | XADD mystream * id 1 msg hello<br>XREAD COUNT 1 STREAMS mystream 0 | id: 1 msg: hello<br>id: 2 msg: hi<br>... |

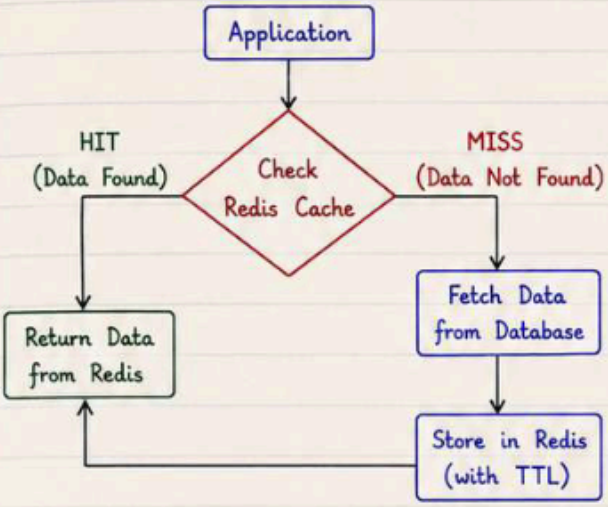
QUICK COMPARISON

| DATA TYPE | BEST FOR | DUPLICATES | ORDER |
|---|---|---|---|
| String | Cache, Counters, Sessions | Allowed | N/A |
| Hash | User profile, product info, settings | N/A | N/A |
| List | Queue, Stack, Message list, Feed | Allowed | Maintains Order |
| Set | Unique values, Tags, Remove duplicates | Not Allowed | No Order |
| Sorted Set | Leaderboard, Ranking, Top N | Not Allowed | Sorted by Score |
| Bitmap | Flags, Online status, Analytics | N/A | N/A |
| HyperLogLog | Approximate unique count | N/A | N/A |
| Stream | Event streaming, Logs, Pipelines | N/A | Maintains Order |

# 3. REDIS CACHING DEEP DIVE

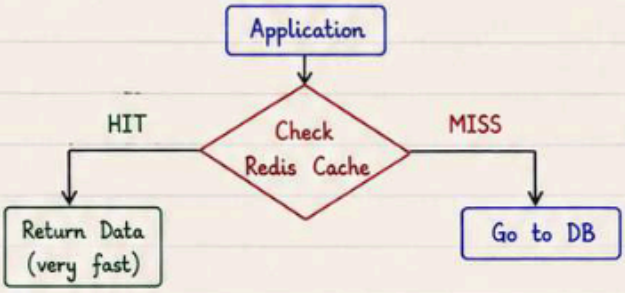
## 1. CACHE ASIDE PATTERN (Most Used)

Application first checks cache (Redis).
If data not found, fetch from DB,
store in cache and return.



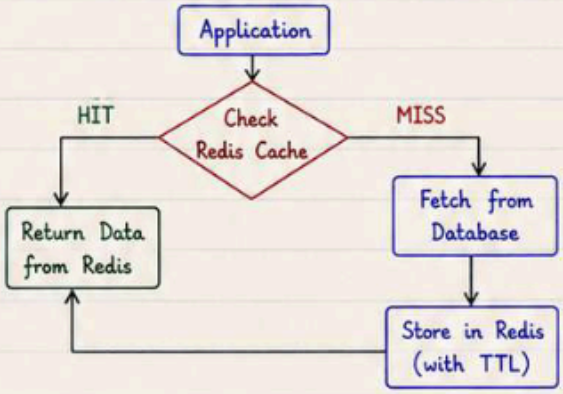

## 2. CACHE HIT

If data is present in Redis,
it returns immediately.




## 3. CACHE MISS

If data not in Redis, fetch from DB,
store in Redis and return.




## 4. TTL (TIME TO LIVE)

Set expiry time for keys so that
stale data is removed automatically.

```
SET user:1 "Aman" EX 60        # expire in 60 sec
SET user:2 "John" PX 5000      # expire in 5000 ms
EXPIRE user:1 120              # set expiry later
TTL user:1                     # check remaining time
PERSIST user:1                 # remove expiry
```

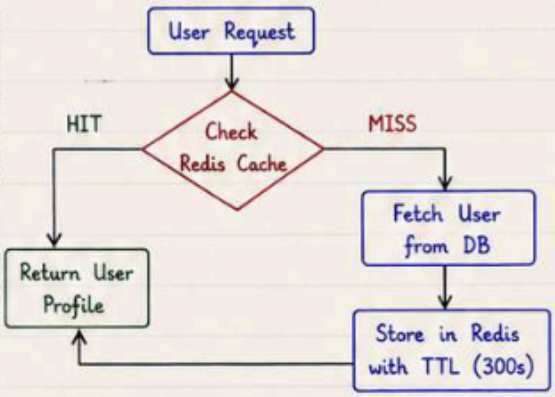
## 5. EVICTION POLICIES

When Redis memory is full, it removes keys based on eviction policy.

| POLICY | DESCRIPTION | USED WHEN |
|---|---|---|
| noeviction | Do not remove any key. Return error. | Not recommended |
| allkeys-lru | Remove least recently used key. | General purpose |
| volatile-lru | Remove least recently used key with expire. | When most keys have TTL |
| allkeys-lfu | Remove least frequently used key. | When some keys are hot |
| volatile-lfu | Remove least frequently used key with expire. | When most keys have TTL |
| volatile-ttl | Remove key with nearest expire time. | Short lived keys |
| volatile-random | Remove random key with expire. | Simple & fast |
| allkeys-random | Remove random key. | Simple & fast |

## 6. EXAMPLE - USER CACHE FLOW

User profile caching example.




## 7. WHEN TO USE CACHE?

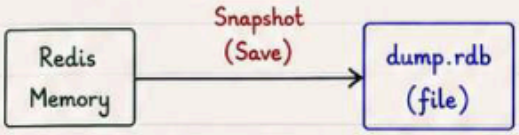
- ✓ Frequently accessed data
- ✓ Read-heavy operations
- ✓ Data that changes infrequently
- ✓ To reduce DB load
- ✓ Improve response time
- ✓ Improve scalability
- ✓ Sessions, Tokens, Counters, Profiles, Product info, etc.

# 4. REDIS PERSISTENCE (Durability)

Redis is in-memory. To avoid data loss on restart or crash, Redis provides persistence options.

## 1. RDB (SNAPSHOT)

- Saves a snapshot of data at a specific time.
- Compact file, fast to load.
- Good for backups.




**How it works?**

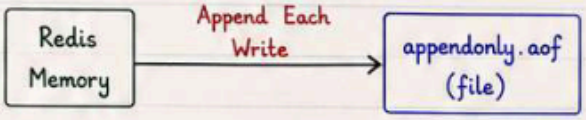
- Redis takes a snapshot and saves to disk.
- On restart, Redis loads snapshot to memory.
- Snapshot times are configurable.

### RDB Save Triggers

| CONFIG (snapshot rule) | MEANING |
|---|---|
| save 900 1 | If 1 key changed in 900 sec (15 min) |
| save 300 10 | If 10 keys changed in 300 sec (5 min) |
| save 60 10000 | If 10000 keys changed in 60 sec (1 min) |
| save "" | Disable RDB snapshots |
| SAVE | Manual save (blocking) |
| BGSAVE | Manual save (background - non-blocking) |

## 2. AOF (APPEND ONLY FILE)

- Logs every write operation in append-only file.
- More durable than RDB.
- Slightly larger file size.




**How it works?**

- Every write command is appended to file.
- On restart, Redis replays commands from AOF.
- Never loses acknowledged writes.

### AOF FSYNC OPTIONS

| OPTION | DESCRIPTION |
|---|---|
| appendfsync always | Sync after every write (safest, slowest) |
| appendfsync everysec | Sync every second (default, balanced) |
| appendfsync no | OS decides when to sync (fastest, risk) |

## 3. RDB vs AOF

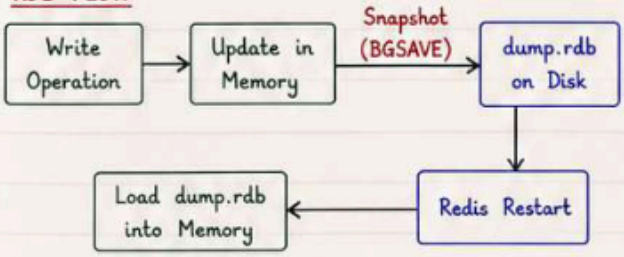
| FEATURE | RDB (SNAPSHOT) | AOF (APPEND ONLY FILE) | BEST FOR |
|---|---|---|---|
| Persistence Type | Snapshot | Logs of write operations | Both are good |
| File Name | dump.rdb | appendonly.aof | - |
| Performance | Faster | Slightly slower | - |
| Data Safety | Can lose recent data | Very safe | AOF |
| File Size | Smaller | Larger | RDB |
| Recovery Time | Faster | Slower (replay logs) | RDB |
| Use Case | Backups, fast recovery | High durability, minimal data loss | AOF |

**RECOMMENDED SETUP**

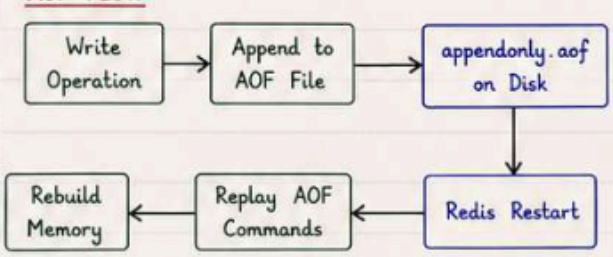
- Enable both RDB + AOF.
- Redis will use AOF for durability and RDB for fast recovery.
- If AOF is enabled, it is the default source of truth.

## 4. PERSISTENCE FLOW

### RDB FLOW




### AOF FLOW



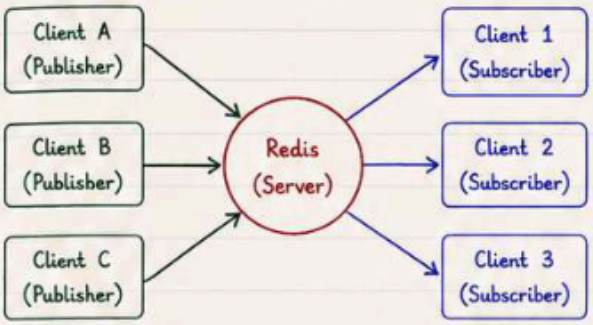

## 5. IMPORTANT NOTES

- ✓ RDB is good for backups and fast recovery.
- ✓ AOF is good for maximum durability.
- ✓ Use 'BGSAVE' not 'SAVE' in production.
- ✓ Regular backups of dump.rdb and appendonly.aof files.
- ✓ Monitor persistence file size and disk space.

# 5. REDIS ADVANCED FEATURES

## 1. PUBLISH / SUBSCRIBE (PUB/SUB)

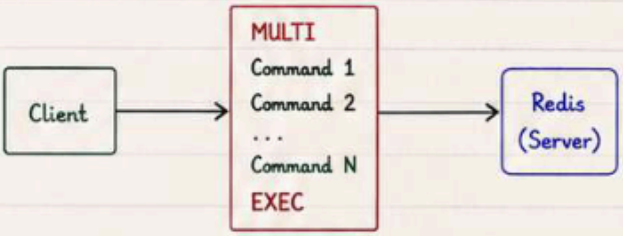
Redis supports real-time messaging.
Used in chat apps, live notifications, events, etc.




- Publishers send messages to a Channel.
- Redis delivers the message to all Subscribers of that Channel.
- Messages are not stored. (Fire and Forget)

## 2. REDIS TRANSACTIONS

Redis transactions are atomic.
All commands in a transaction are executed sequentially.

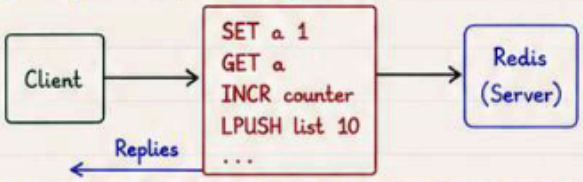



- MULTI → Marks the beginning
- EXEC → Executes all commands
- DISCARD → Cancels the transaction

Note: Redis transactions do not support rollback.
If an error occurs, remaining commands still run.

## 3. REDIS PIPELINING

Send multiple commands together without waiting for replies. Improves performance.




- Commands are queued and sent in a batch.
- Redis processes and returns all replies together.
- Reduces network round trips.

## 4. REDIS SCAN FAMILY

Used to iterate keys without blocking Redis.

- KEYS * → Returns all keys (Blocks Redis)
- SCAN cursor [MATCH pattern] [COUNT n]
- HSCAN key cursor
- SSCAN key cursor
- ZSCAN key cursor

SCAN is incremental. It returns small batches of keys using a cursor.

## COMPARISON

| FEATURE | PUB/SUB | TRANSACTIONS | PIPELINING | SCAN FAMILY |
|---|---|---|---|---|
| Purpose | Real-time messaging | Atomic execution | Performance (Batching) | Iterate keys safely |
| Data Stored | No (Fire & Forget) | Yes | Yes | N/A |
| Blocking | No | No | No | No (Non-blocking) |
| Use Case | Chat, Notifications | Banking, Counters | High throughput apps | Admin, Reports, Cleanup |

## 5. REAL-WORLD USE CASES

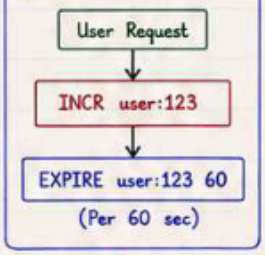
### 1. LEADERBOARD (Sorted Set)

Store scores and get top N players.

```
ZADD leaderboard
  100 Ronak
  200 Aman
  150 John
...
ZREVRANGE leaderboard
0 9 WITHSCORES
```

### 2. RATE LIMITING (INCR + EXPIRE)

Limit API calls per user.



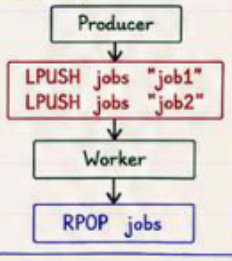

### 3. SESSION STORE (Hash)

Store user session data.

```
HSET session:1001
  userId    "Aman"
  role      "admin"
  lastLogin "10:30"
  ...
EXPIRE session:1001 3600
```

### 4. JOB QUEUE (List)

Push jobs and workers consume.




### 5. REAL-TIME ANALYTICS (Counters)

Track events, views, clicks, etc.

```
INCR page:views
INCR user:signups
INCR product:clicks
...

GET page:views
```

## QUICK COMMAND CHEAT SHEET

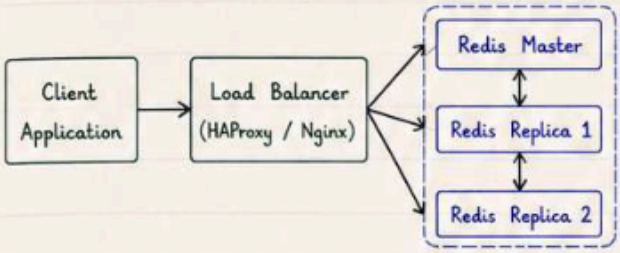
```
PUBLISH channel message     # Send message to channel
SUBSCRIBE channel           # Listen to channel
MULTI                       # Start transaction
EXEC                        # Execute transaction
DISCARD                     # Cancel transaction

PIPELINE (client side)      # Send batch of commands
SCAN cursor MATCH pattern COUNT n  # Incremental key scan
HSCAN key cursor            # Scan hash keys
SSCAN key cursor            # Scan set members
ZSCAN key cursor            # Scan sorted set members
```

# 6. REDIS IN PRODUCTION - BEST PRACTICES & CHEAT SHEET

## 1. REDIS DEPLOYMENT OPTIONS

- Standalone → Single Redis Server
- Master - Replica → Replication for high availability.
- Cluster Mode → Sharding + High availability




- Clients connect to Master.
- Replicas are used for read scaling & failover.
- If Master fails, Replica is promoted.

## 2. REDIS CLUSTER (SHARDING)

- Data is split across multiple Masters (Slots).
- Each Master can have Replicas.
- Automatic failover and rebalancing.




- Total 16384 slots.
- Clients connect to any node.
- Cluster handles routing.
- Use redis-cli --cluster create for setup.

## 3. SECURITY BEST PRACTICES

- ✓ Set strong password (requirepass)
- ✓ Bind to specific IP (bind 127.0.0.1 / private IP)
- ✓ Disable dangerous commands (FLUSHALL, KEYS *)
- ✓ Use TLS/SSL for encrypted communication
- ✓ Use ACL (Access Control Lists)
- ✓ Keep Redis updated

Example - Disable Dangerous Commands

```
rename-command FLUSHALL ""
rename-command KEYS ""
```

## 4. PERFORMANCE TIPS

- Use appropriate data structure.
- Set TTL for temporary data.
- Avoid storing huge values.
- Use Pipelining to reduce RTT.
- Use connection pooling.
- Monitor memory and slow logs.
- Use Replica for read-heavy workloads.

## 5. MONITORING & TOOLS

- INFO → Server metrics
- MONITOR → Real-time commands
- SLOWLOG → Slow queries
- redis-cli --latency → Latency test
- MEMORY USAGE <key> → Key memory
- redis-benchmark → Load test

Useful Metrics (INFO)

- used_memory
- connected_clients
- instantaneous_ops_per_sec
- keyspace_hits / keyspace_misses
- expired_keys

## 6. REDIS vs MEMCACHED

| FEATURE | REDIS | MEMCACHED |
|---|---|---|
| Data Structures | Many (String, List, Set, Hash, etc.) | Only String (Key-Value) |
| Persistence | Yes (RDB, AOF) | No |
| Replication | Yes (Master-Replica) | No |
| Clustering | Yes (Cluster Mode) | No |
| Transactions | Yes | No |
| Memory Management | Advanced (Eviction, TTL, etc.) | Simple (LRU) |
| Use Case | Caching, Queues, Counters, Sessions, etc. | Caching only |
| Speed | Slightly slower (more features) | Faster (simple) |

## 7. COMMON USE CASES

- Caching (Web, API, DB Queries)
- Session Management
- Rate Limiting
- Leaderboard / Ranking
- Counters (Page views, Likes, etc.)
- Job Queue / Background Jobs
- Pub/Sub (Chat, Notifications)
- Real-time Analytics
- Distributed Locks

## 8. REDIS COMMAND CHEAT SHEET

| STRING | LIST | SET | HASH | SORTED SET (ZSET) | OTHER |
|---|---|---|---|---|---|
| SET key value | LPUSH key value | SADD key m1 m2 | HSET key field val | ZADD key score mem | EXPIRE key sec |
| GET key | RPUSH key value | SMEMBERS key | HGET key field | ZRANGE key 0 -1 | TTL key |
| INCR key | LPOP key | SREM key m1 | HGETALL key | ZREVRANGE key 0 -1 | DEL key |
| DECR key | RPOP key | SISMEMBER key m1 | HDEL key field | ZINCRBY key incr mem | EXISTS key |
| MSET k1 v1 k2 v2 | LRANGE key 0 -1 | SCARD key | HKEYS key | ZREM key mem | KEYS pattern |
| MGET k1 k2 | LLEN key | | HVALS key | ZSCORE key mem | FLUSHDB |

## 9. QUICK RECAP

- ✓ Redis is in-memory and super fast.
- ✓ Choose the right data structure.
- ✓ Use TTL and eviction wisely.
- ✓ Enable persistence for reliability.
- ✓ Secure your Redis instance.
- ✓ Monitor and optimize regularly.
- ✓ Use clustering and replication for high availability.
- ✓ Redis is not just a cache - it's a powerful tool!

**Practice + Build + Deploy = Master Redis**